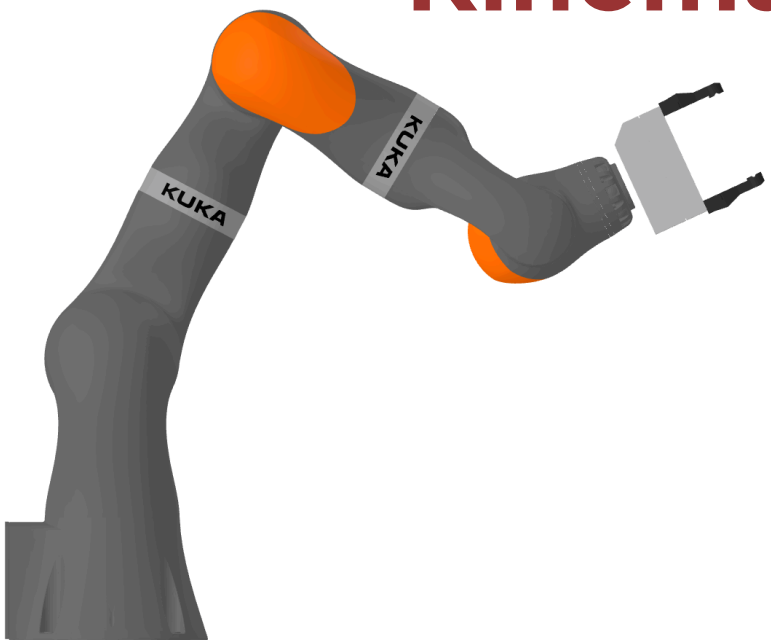
Kinematics via Optimization



MIT 6.4210/2: Robotic Manipulation

Fall 2026, Lecture 5

Massachusetts Institute of Technology

Kinematics

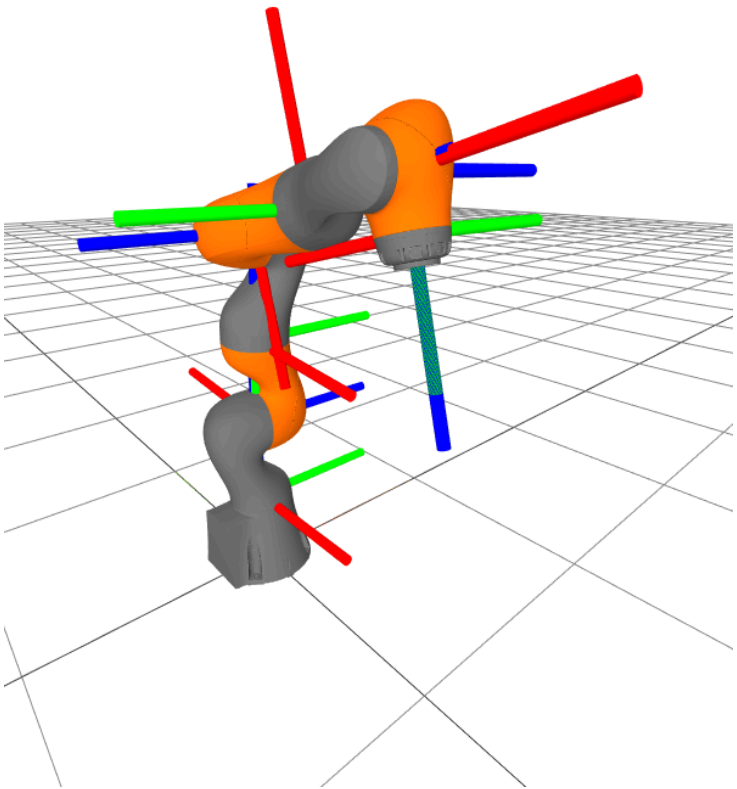
- *Forward kinematics*: joint positions ⇒ pose $q \Rightarrow X^B$
- *Inverse kinematics**: pose ⇒ joint positions $X^B \Rightarrow q$

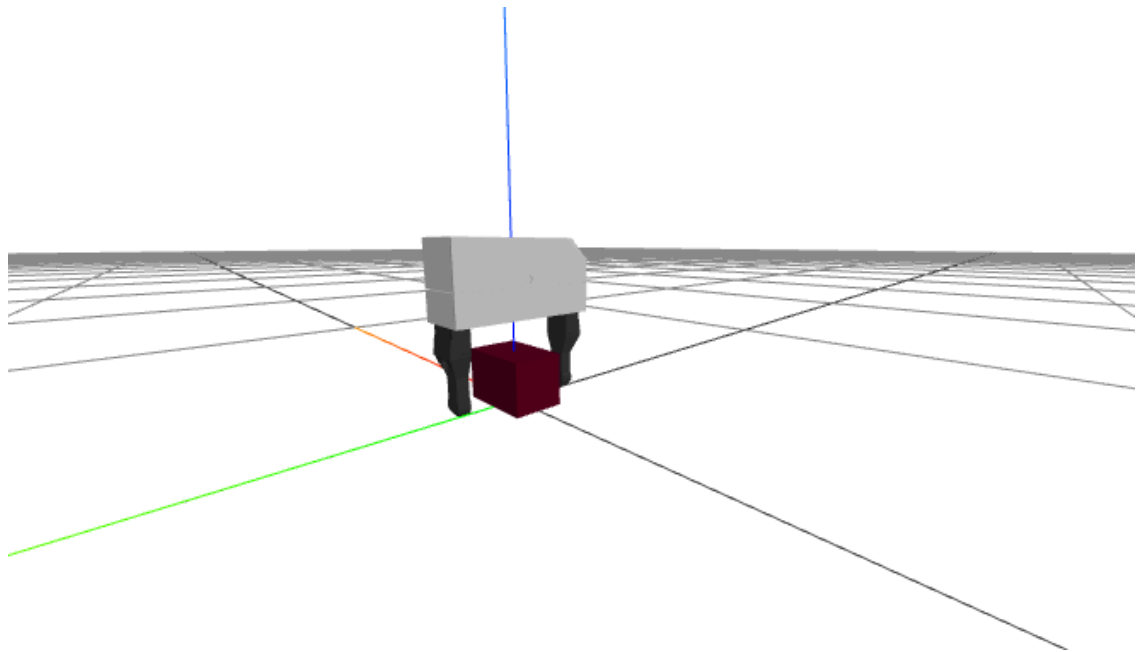
- *Differential kinematics*:
  joint positions, velocities ⇒ spatial velocity $q, v \Rightarrow V^B$

- *Differential **inverse** kinematics*:
  spatial velocity, joint positions ⇒ joint velocities
  $V^B, q \Rightarrow v$

Basic Pick and Place

Step 1: Kinematic Frames / Spatial Algebra

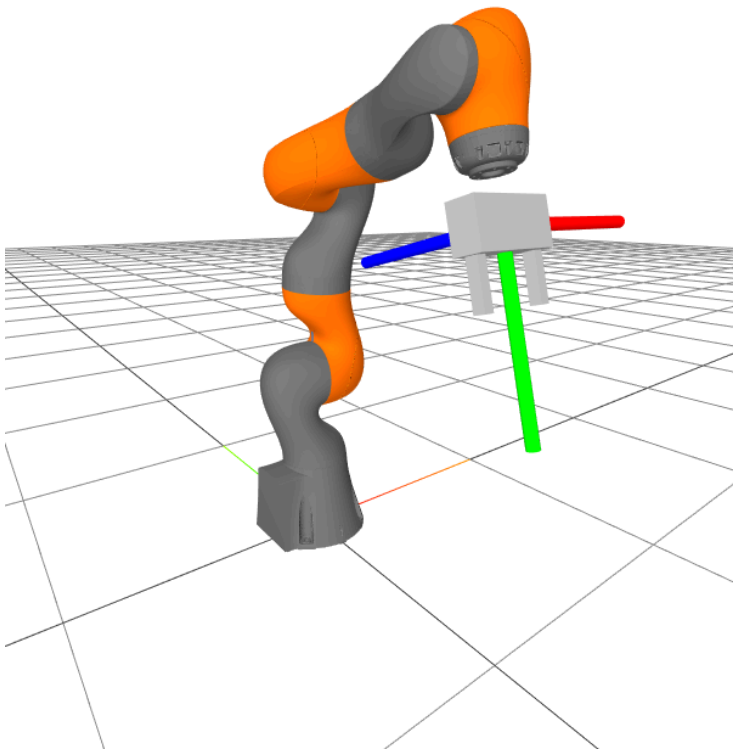


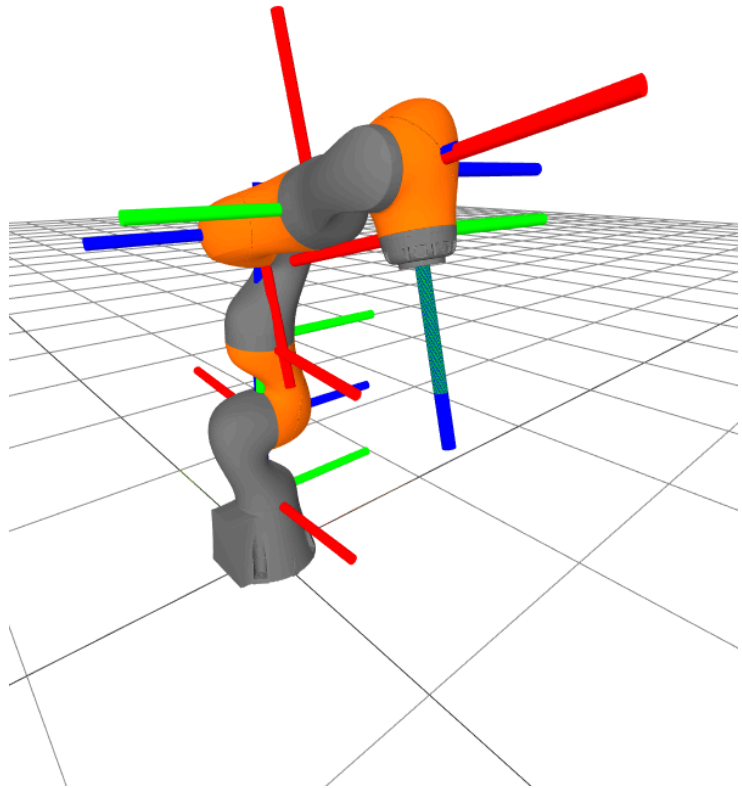


Step 2: Gripper Frame Plan "Sketch"

https://manipulation.mit.edu/data/pick_sketch.html

Step 3: Forward kinematics of the iiwa + WSG





Step 4: Differential Inverse Kinematics

Kinematic Jacobian: $^{W}V^{G} = J^{G}(q)v$

spatial velocity

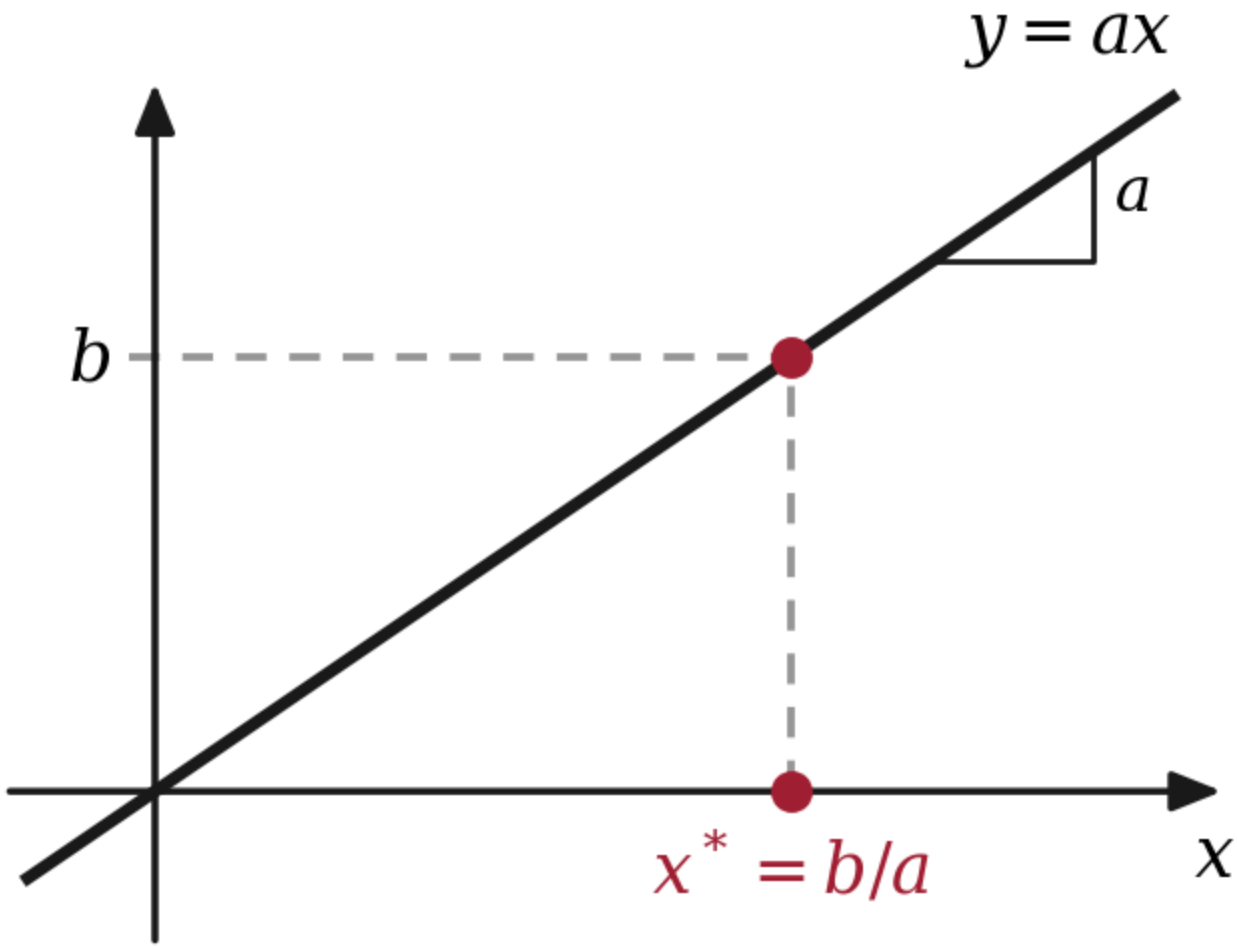
generalized velocity



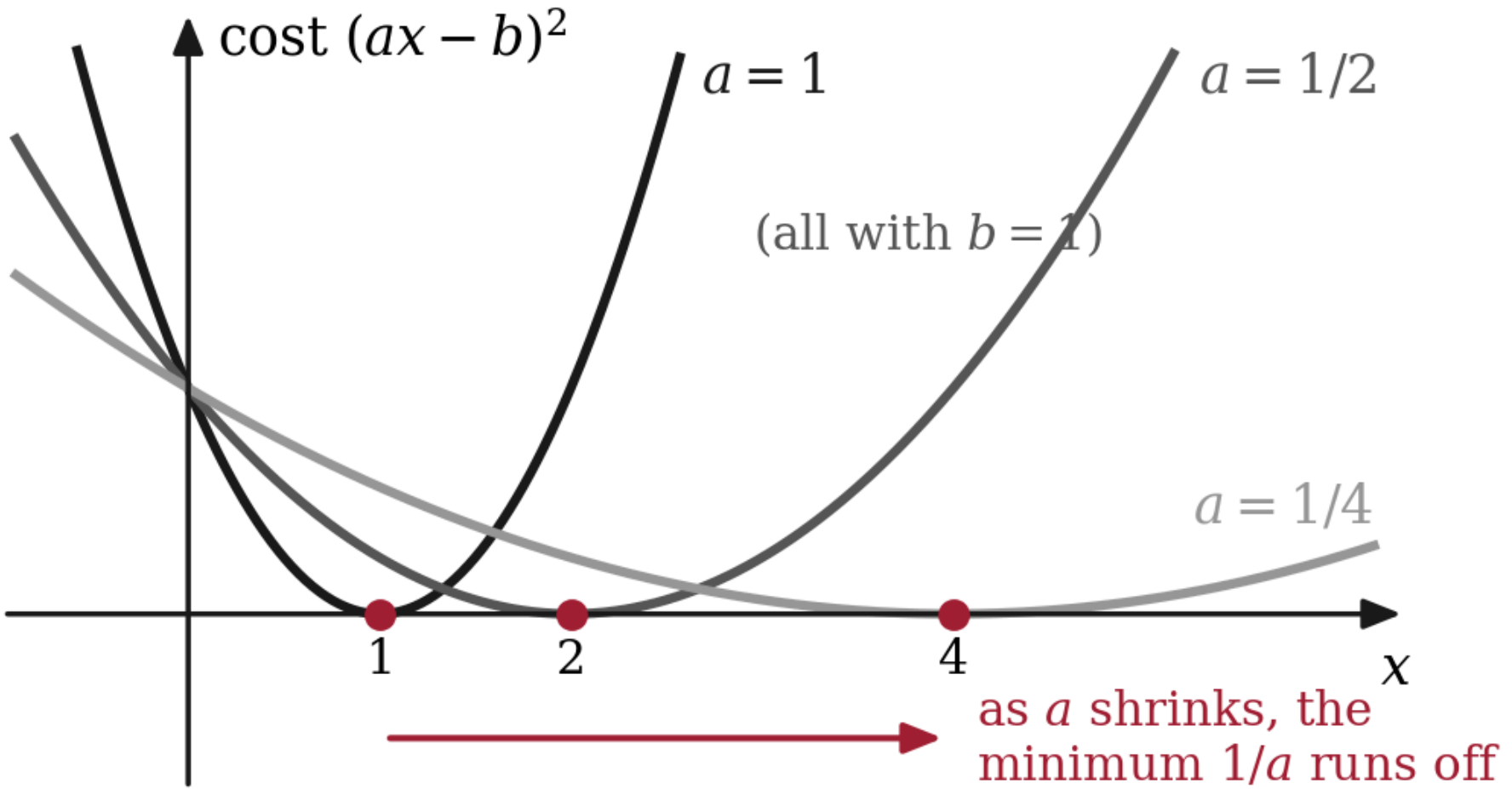
$$v = [J^{G}(q)]^{+W}V^{G}$$

https://manipulation.mit.edu/pick.html#diff_ik_w_constraints

Scalar case – equality

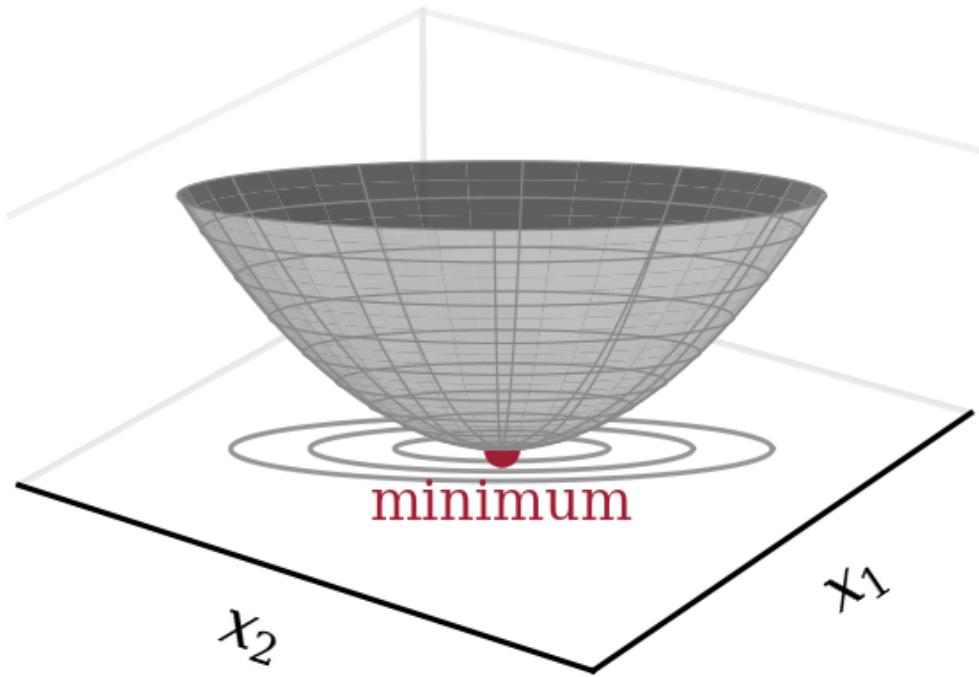


Scalar case - approximately

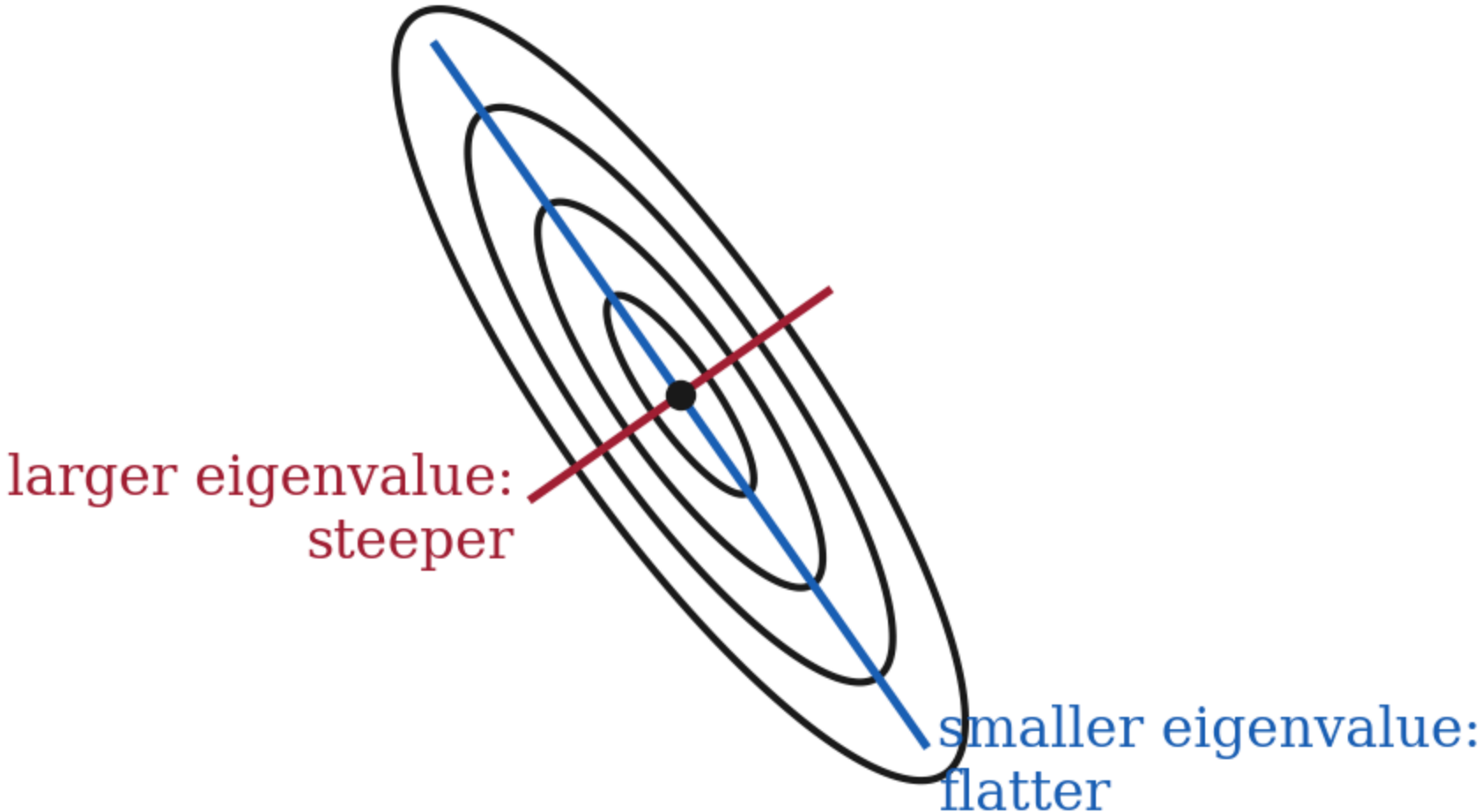


The matrix case

$$\|Ax - b\|^2$$



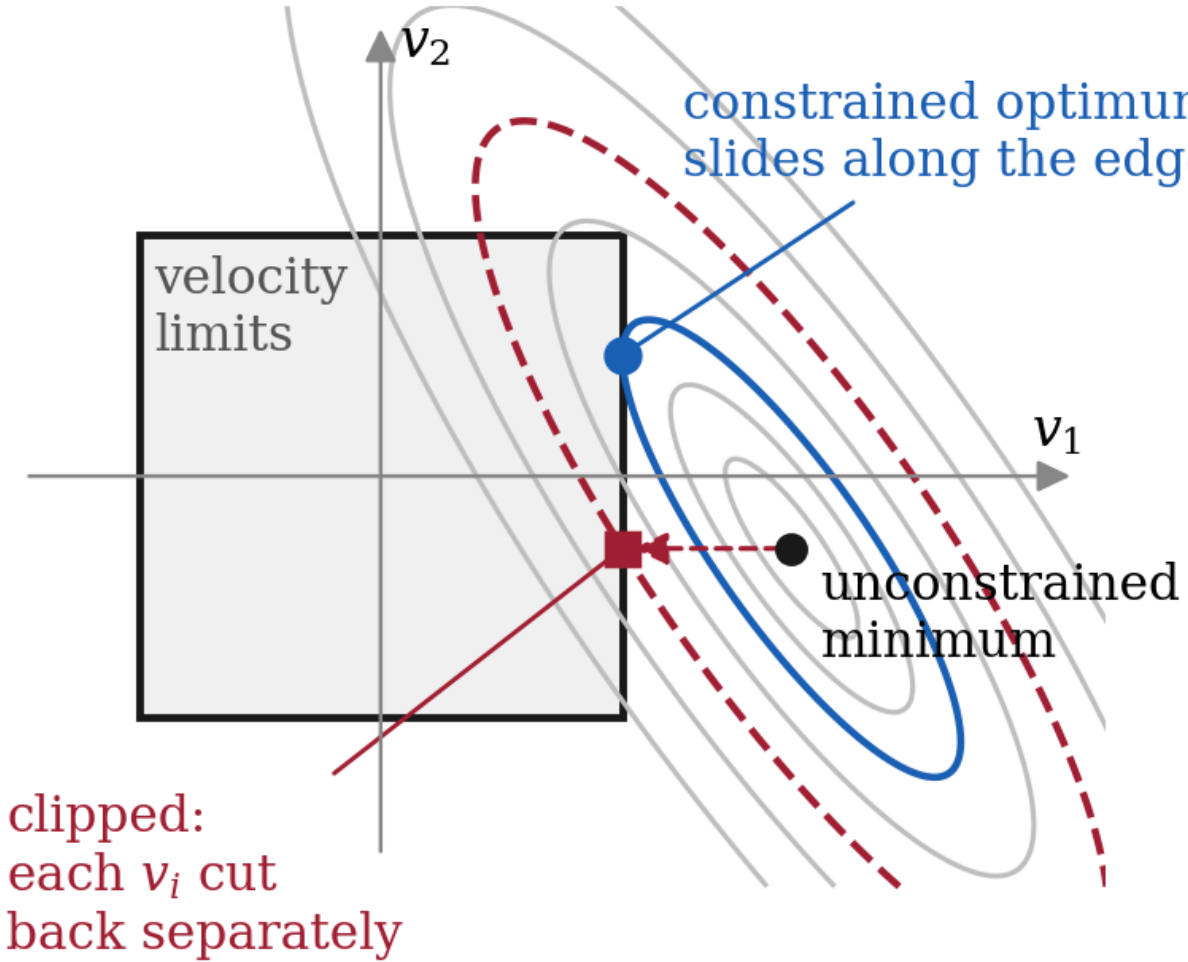
Level sets of cost



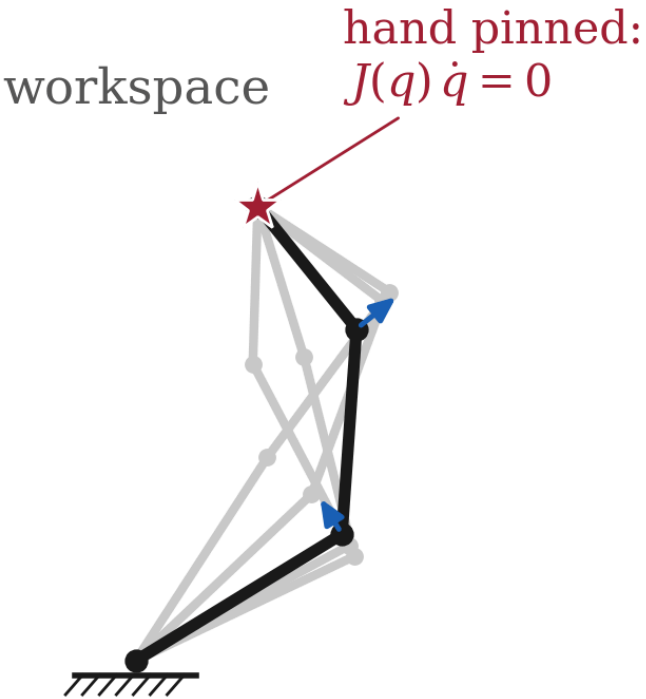



```
1 prog = MathematicalProgram()
2 x = prog.NewContinuousVariables(2)
3 prog.AddConstraint(x[0] + x[1] == 1)
4 prog.AddConstraint(x[0] <= x[1])
5 prog.AddCost(x[0] ** 2 + x[1] ** 2)
6 result = Solve(prog)
```

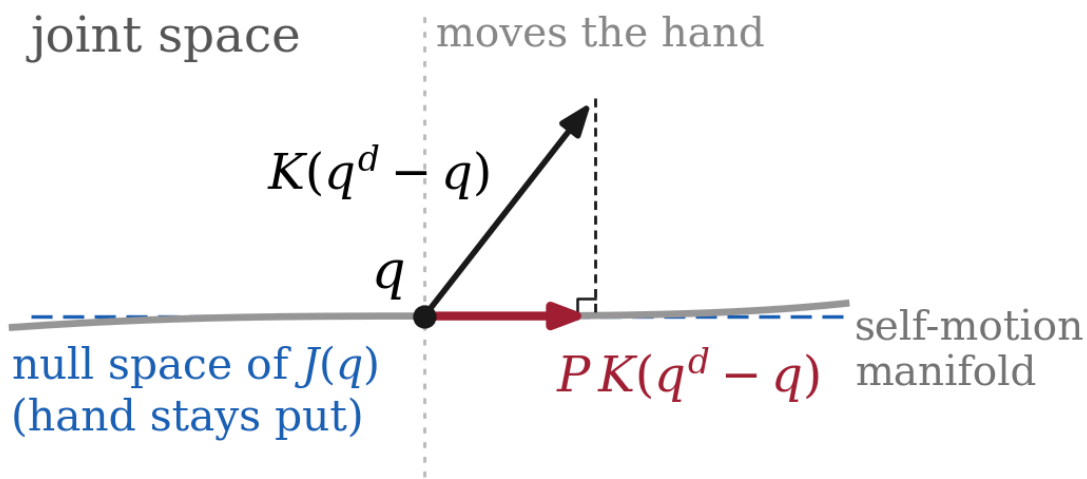
Clipping vs constraints

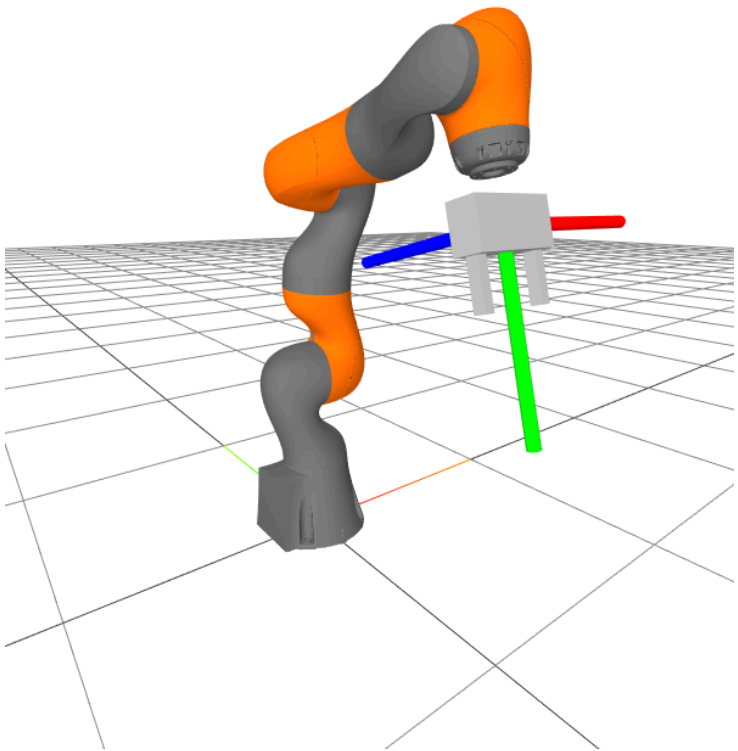


Joint centering in null space of J



self-motion: the joints move, the hand does not





$$X^B = f_{kin}^B(q)$$

Inverse kinematics (a rich history!)

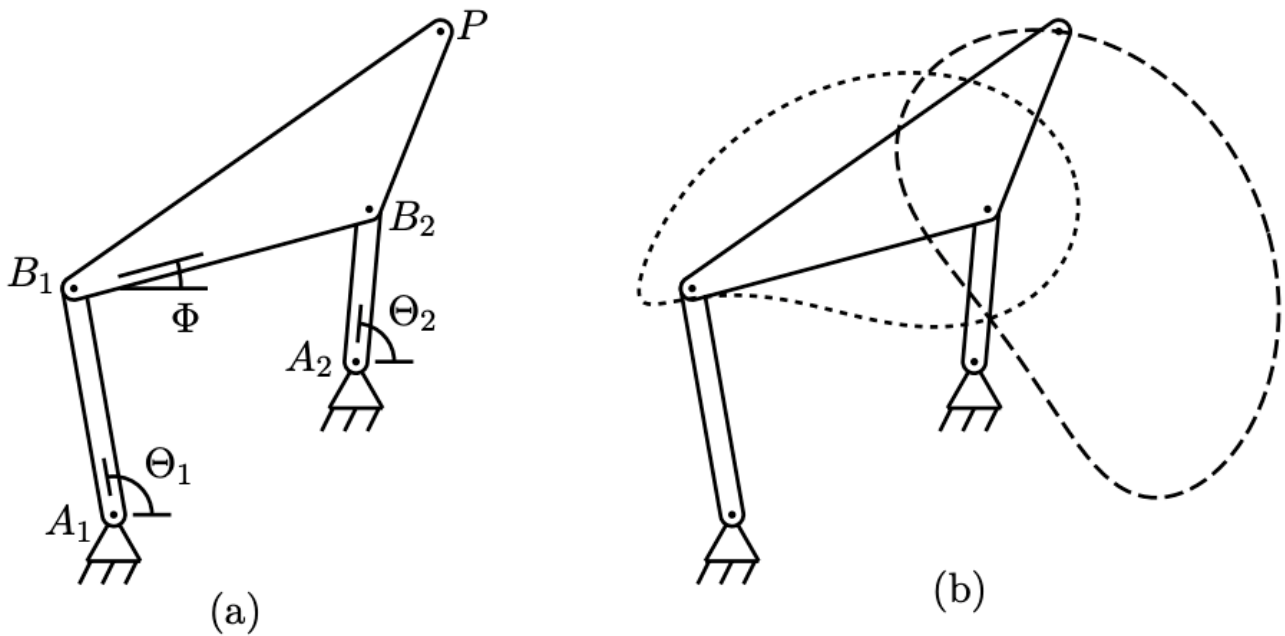
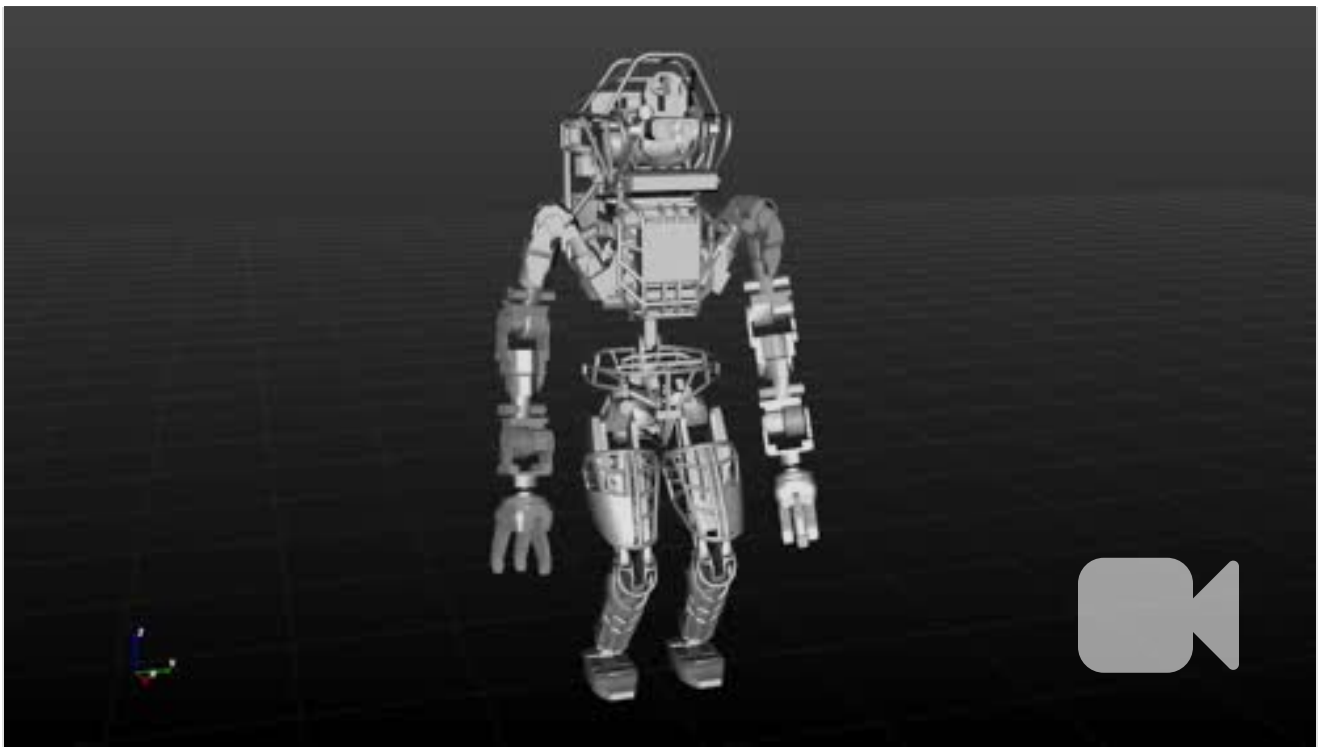


Figure 4: Four-bar linkage.

Charles W. Wampler and Andrew J. Sommese. Numerical algebraic geometry and algebraic kinematics. Acta Numerica, 20:469-567, 2011.

Inverse kinematics as an optimization



$$\min_q |q - q_{desired}|$$

subject to:

- rich end-effector constraints
- joint limits
- collision avoidance
- "gaze constraints"
- "feet stay put"
- balance (center of mass)
- ...

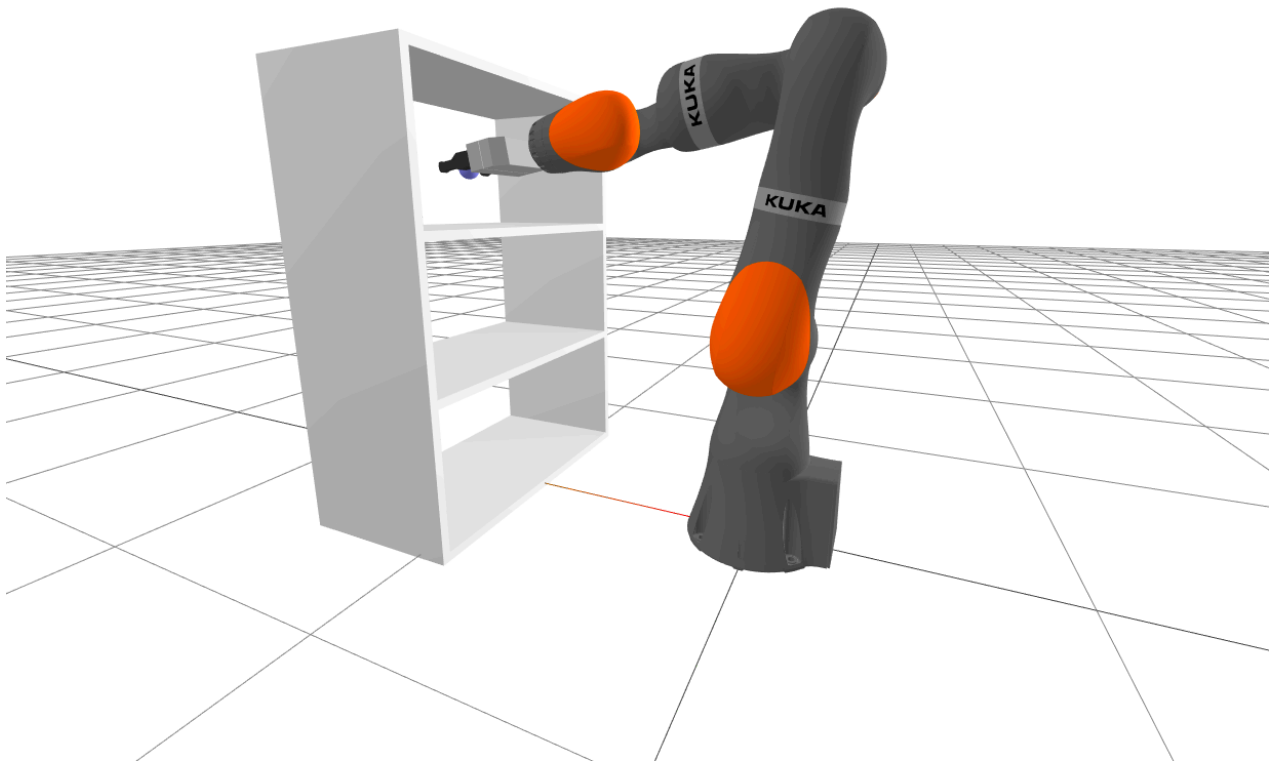
https://drake.mit.edu/doxygen_cxx/classdrake_1_1multibody_1_1_inverse_kinematics.html

```
        ik = InverseKinematics(plant, plant_context)
        ik.AddPositionConstraint(
            gripper_frame,
            [0, 0, 0],
            plant.world_frame(),
            pose.translation(),
            pose.translation(),
        )
        ik.AddOrientationConstraint(
            gripper_frame,
            RotationMatrix(),
            plant.world_frame(),
            pose.rotation(),
            0.0,
        )
        prog = ik.get_mutable_prog()
        q = ik.q()
        prog.AddQuadraticErrorCost(np.identity(len(q)), q0, q)
        prog.SetInitialGuess(q, q0)
        result = Solve(ik.prog())
```

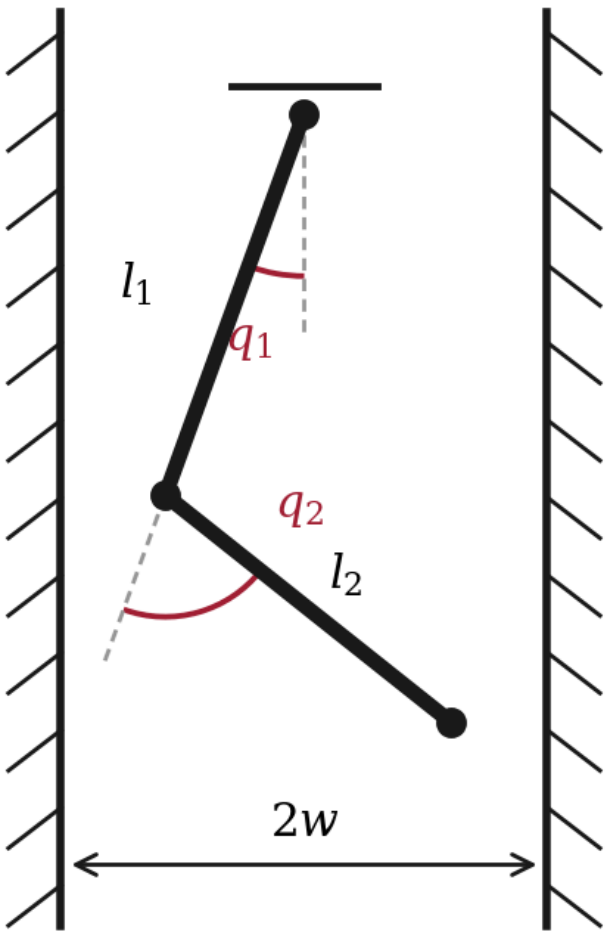
https://drake.mit.edu/doxygen_cxx/group__solver__evaluators.html

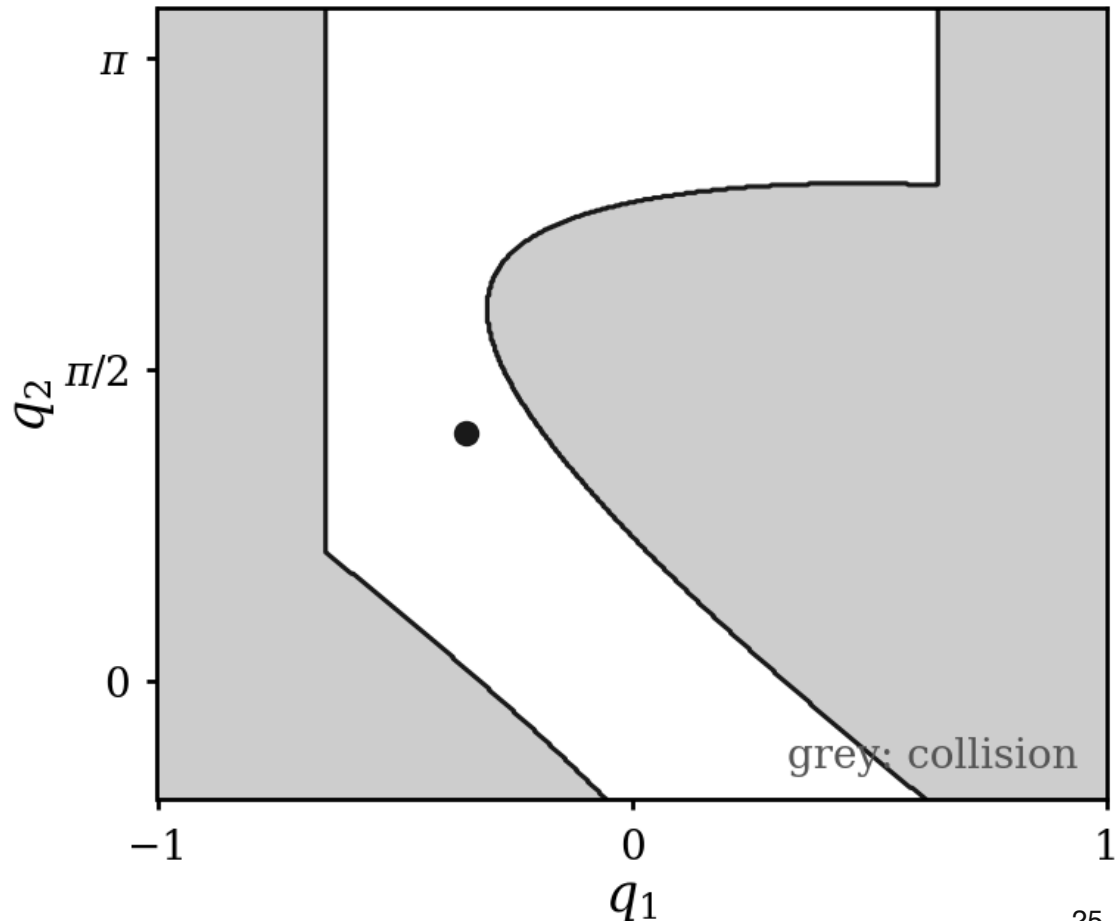
https://www.youtube.com/embed/E_CVq0lWfSc?enablejsapi=1&mute=1&rel=0

https://drake.mit.edu/doxygen_cxx/classdrake_1_1multibody_1_1_minimum_distance_lower_bound_constraint.html


KUKA
KUKA

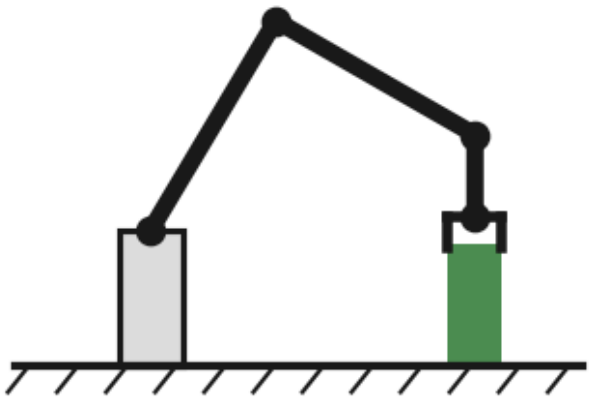
Configuration space



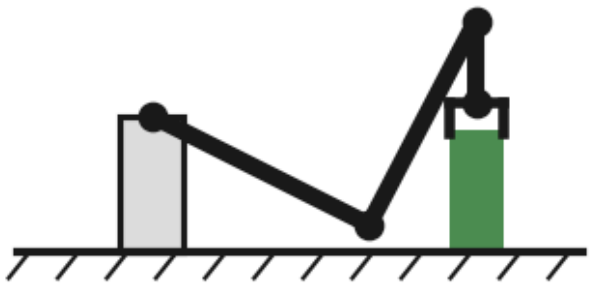


https://www.desmos.com/calculator/kol7mvpgke

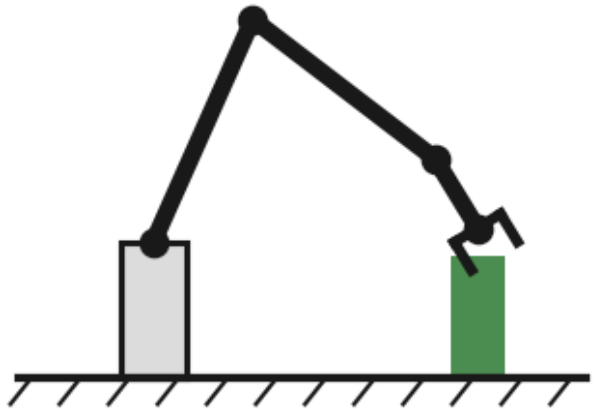
One grasp - three solutions



elbow up



elbow down

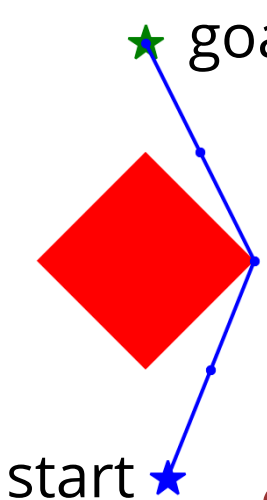


tilted approach

https://manipulation.csail.mit.edu/data/iiwa_shelves_configuration_space.html

https://manipulation.csail.mit.edu/data/shelf_ik_prog.html

Motion planning as a (nonconvex) optimization



fixed number of samples

$$\min_{q_0,\ldots,q_N} \sum_{n=0}^{N-1} |q_{n+1} - q_n|_2^2$$

$$\text{subject to} \quad q_0 = q_{start}$$

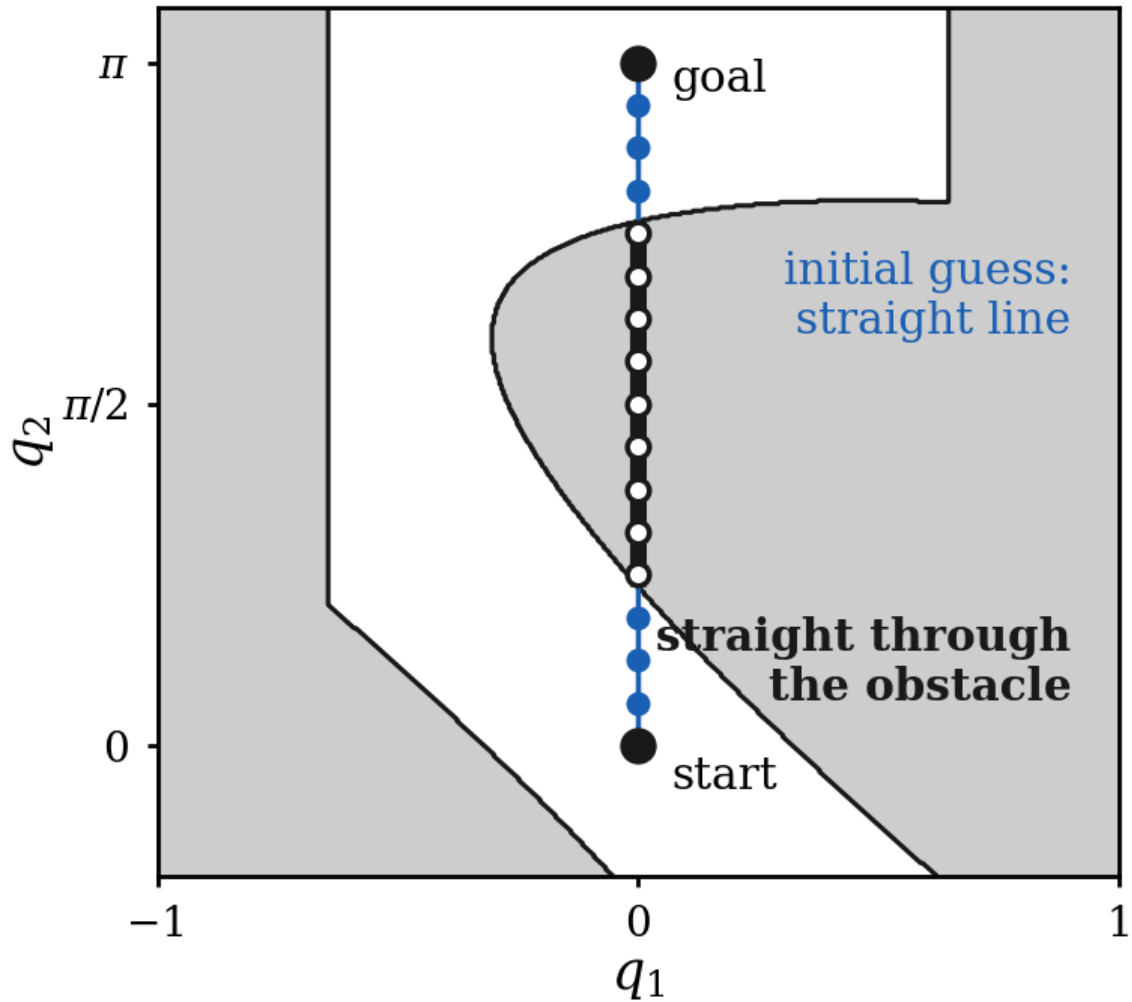
$$q_N = q_{goal}$$

collision-avoidance → $|q_n|_1 \geq 1 \quad \forall n$

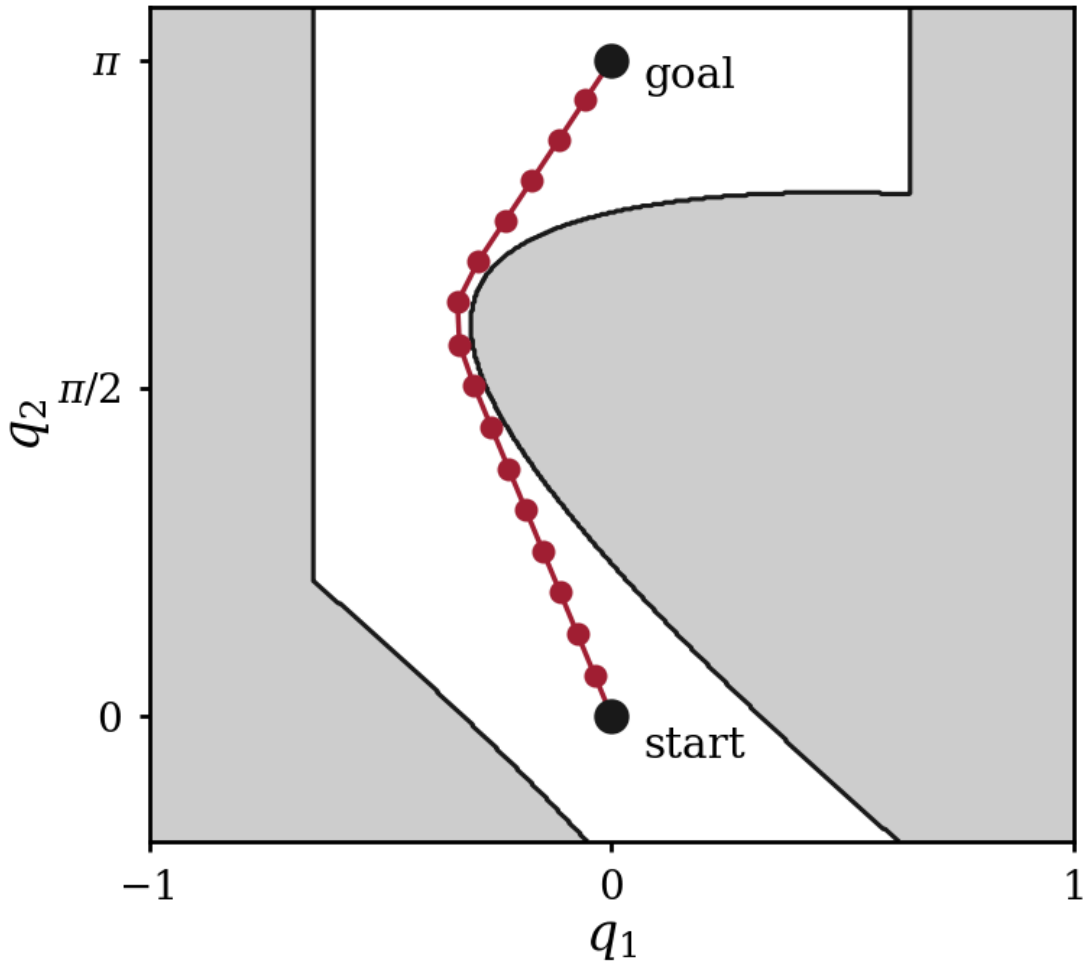
(outside the L^1 ball)

nonconvex

Initial trajectory guess



Optimized trajectory



Motivation: Moving fast! (at the dynamic limits)

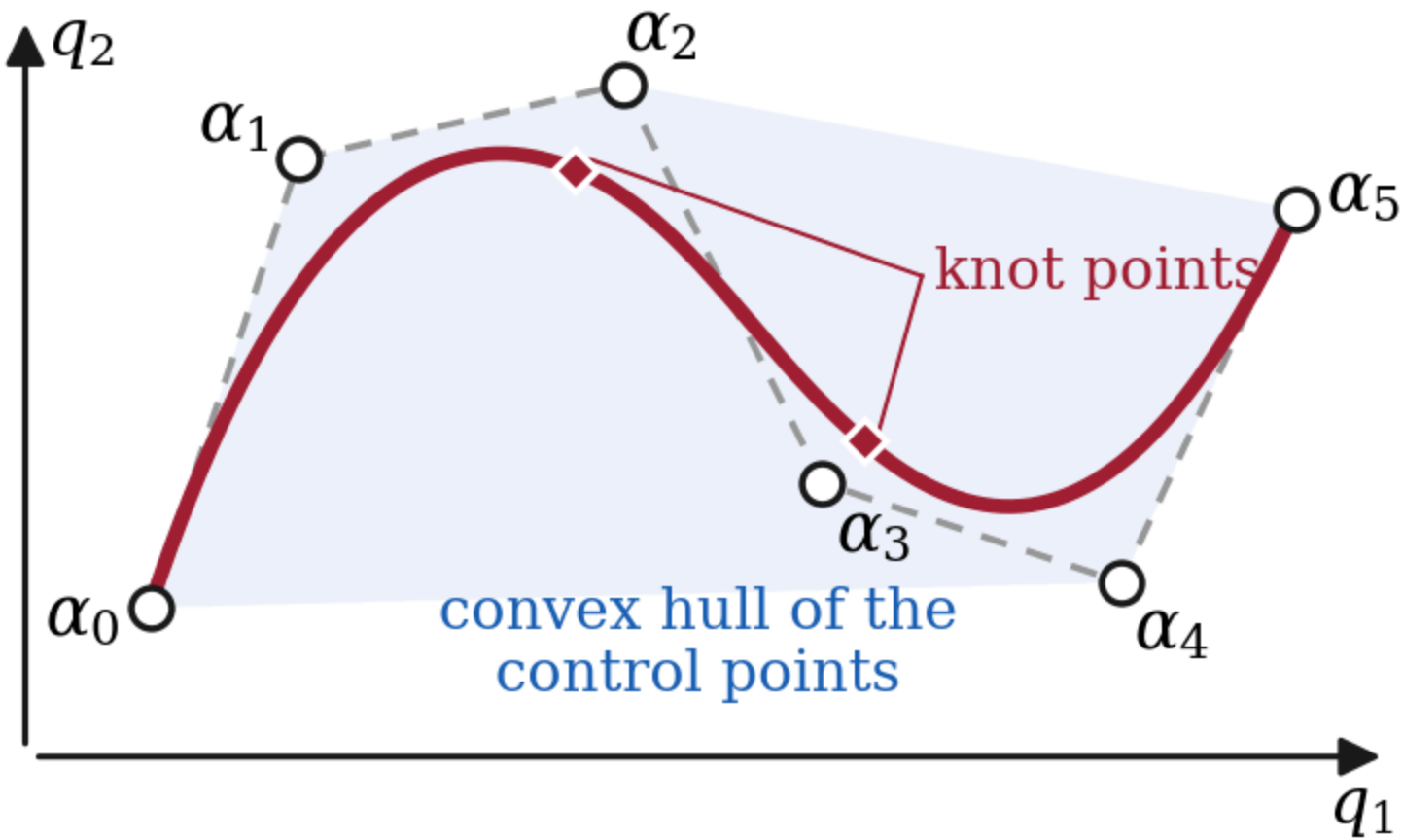


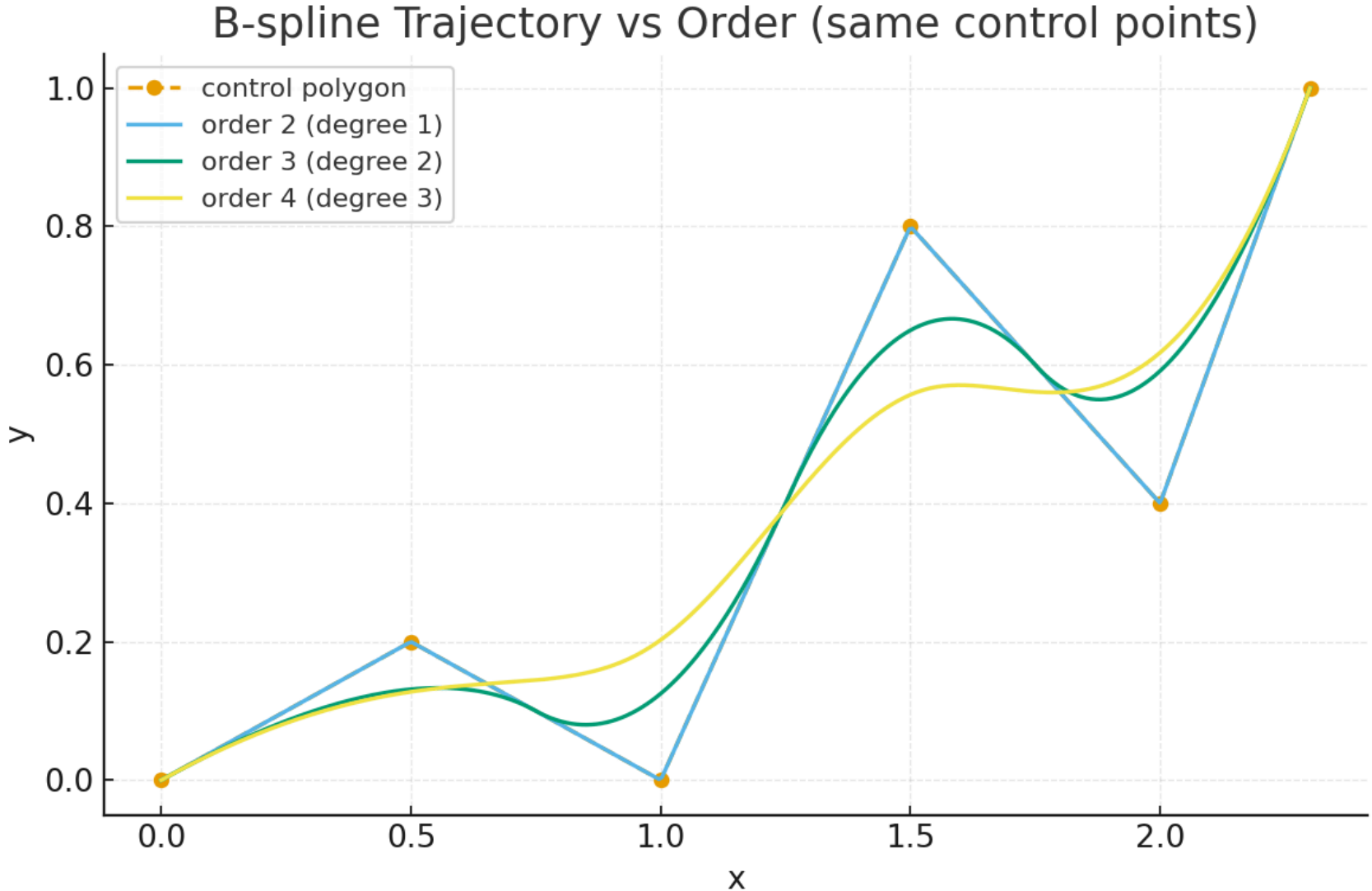


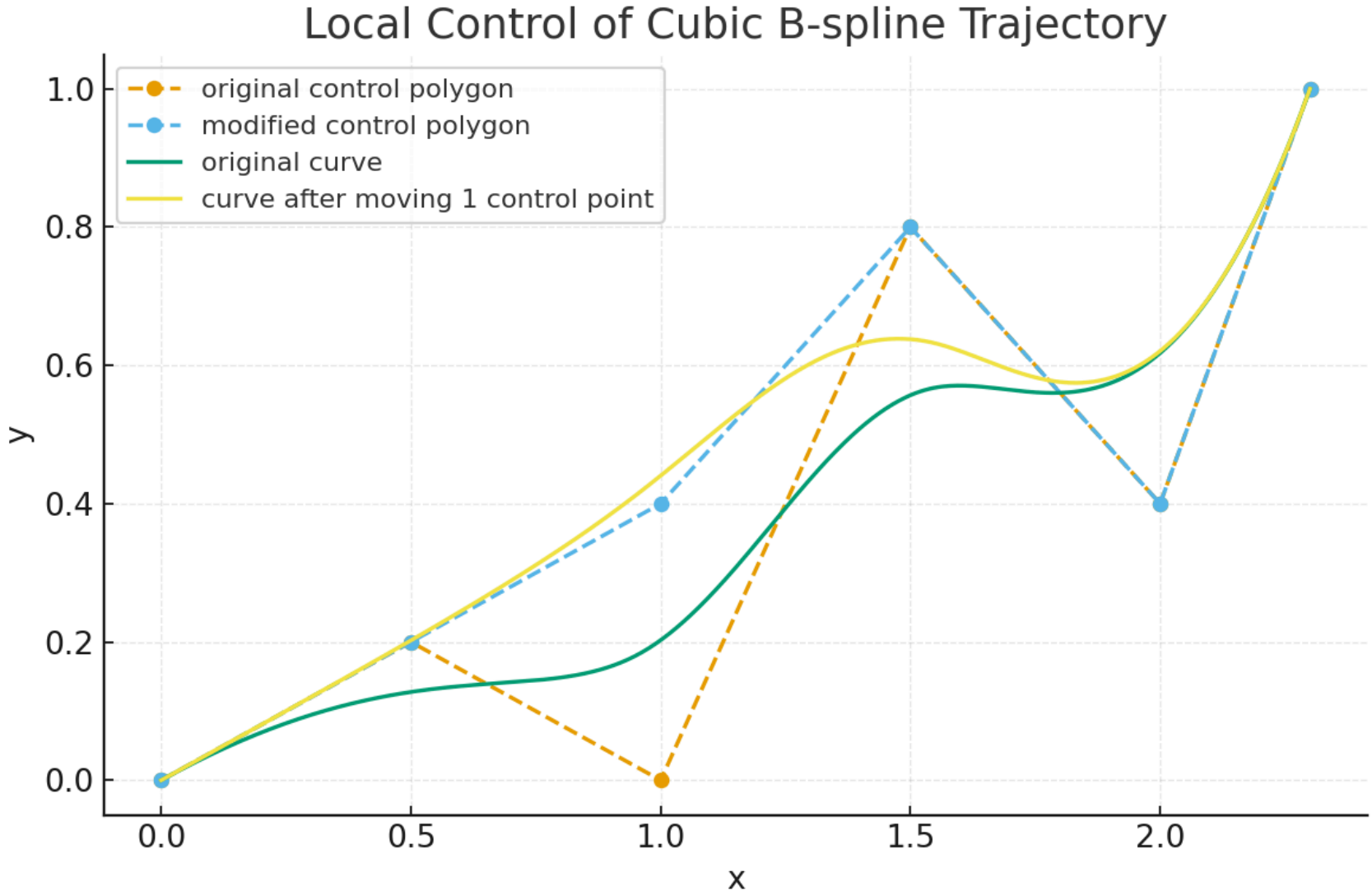
"Using trajectory optimization made scooping up to twice as fast"

credit: DEXAI ROBOTICS

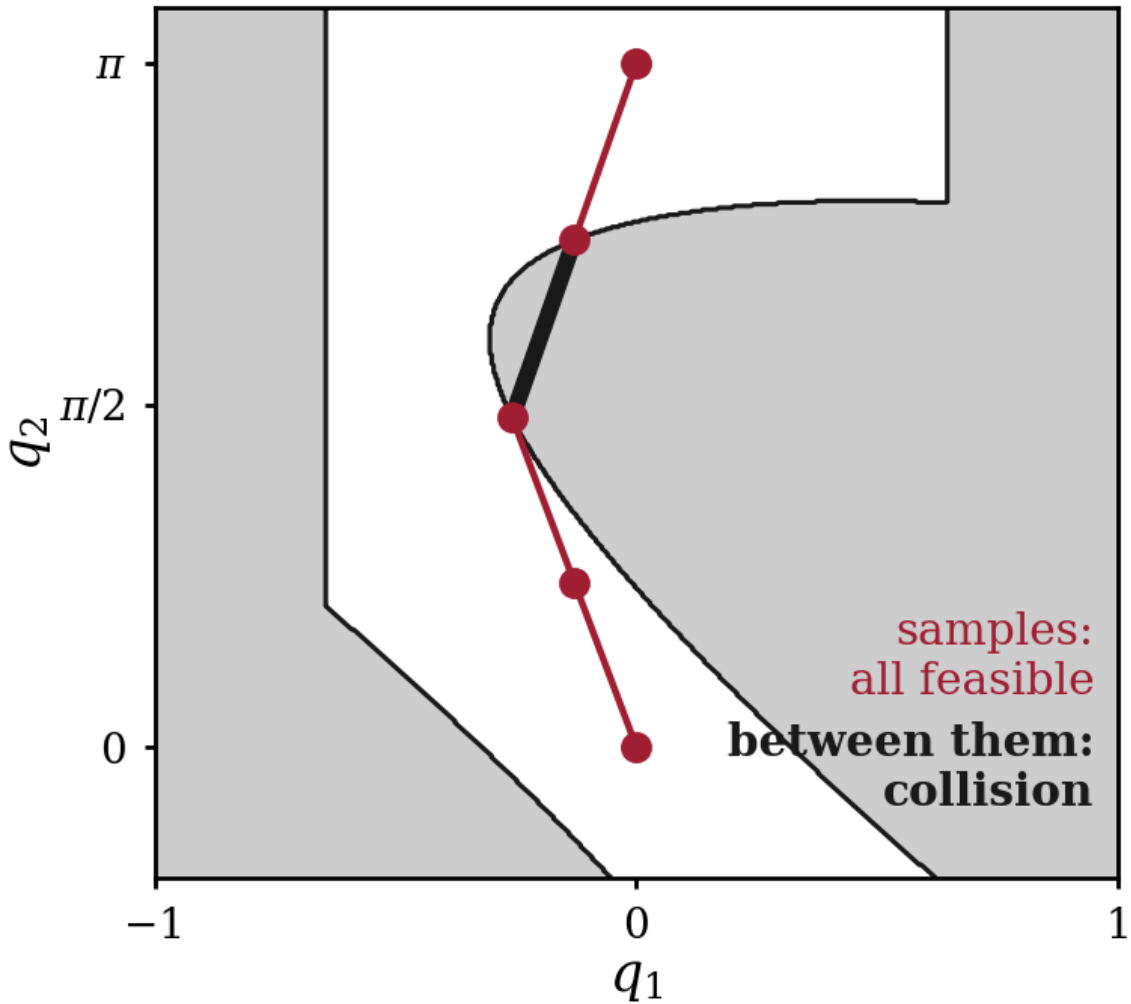
Cubic B-spline




B-spline Trajectory vs Order (same control points)
control polygon
order 2 (degree 1)
order 3 (degree 2)
order 4 (degree 3)
1.0
0.8
0.6
0.4
0.2
0.0
y
0.0
0.5
1.0
1.5
2.0
x


Local Control of Cubic B-spline Trajectory
original control polygon
modified control polygon
original curve
curve after moving 1 control point
1.0
0.8
0.6
0.4
0.2
0.0
y
0.0
0.5
1.0
1.5
2.0
x

Failure mode



https://manipulation.csail.mit.edu/data/kinematic_trajectory_optimization_karate.html